

AI DATA AGENTS

Four Functional Tool-Using Agents on a Local, Testable Agent Framework

Project 16 of the AI Engineering Portfolio | Build Guide & Interview Companion
Author: Anas Amiar

"The agent loop is just software — testable with a fake model. The agent behavior lives in the system prompt. Split those two and you get agents you can verify offline and run live with nothing but an API key."

How to Use This Guide

This guide documents the project exactly as it was built: a short pitch, the tech stack, a phase-by-phase build order with reasoning for why each phase comes before the next, and the talking points worth memorizing before an interview. Read top to bottom to learn the project from scratch, or jump to the "Interview Talking Point" box at the end for a quick pre-call refresher.

What You're Building

Four autonomous agents — data analyst, data scientist, AI engineer, ML engineer — on a local agent framework: a real Anthropic tool-use loop with sandboxed bash / read_file / write_file tools. Each agent writes and runs actual Python in its own per-run working directory: the analyst produces a plotly HTML report from any CSV; the data scientist delivers a cross-validated model + honest metrics; the AI engineer ships a RAG package with a pytest eval gate; the ML engineer builds a FastAPI service with a Dockerfile and a self-verified test suite.

The roles and system prompts come from Anthropic's Managed Agents cookbooks (notebooks included in the repo); this project makes them runnable locally with just an ANTHROPIC_API_KEY — and the framework itself is verified offline with a scripted FakeClient: **5 passing tests, zero tokens spent**.

WHY THIS PROJECT LANDS INTERVIEWS

Everyone can call an LLM; far fewer can build the loop around it — tool execution, sandboxing, turn-taking, failure handling — and fewer still design it so the loop is testable without paying for live model calls. The FakeClient pattern (dependency-inject the model, script realistic tool sequences) is the same discipline production agent teams use, and it doubles as proof you understand both halves of an agent: the software and the prompt.

Tech Stack

Layer	Tool / Library	Purpose
Language	Python 3.11+	Core implementation
LLM API	Anthropic Messages API	Standard tool-use loop; model configurable
Tools	bash / write_file / read_file	Sandboxed to a per-run working directory
Safety	Timeouts + path-escape rejection	300s bash cap, output truncation, cwd scoping
Testing	pytest + FakeClient	Loop verified offline with a scripted model
Agent definitions	Cookbook system prompts	Baselines, CV, eval gates, self-verification
Sample data	4 generated datasets	sales, churn (planted noise), houses, FAQ
CLI	python3 -m agentkit.run	Any agent, any CSV/markdown, any task

The Core Flow

run.py copies your data into a fresh runs/<agent>-<timestamp>/ directory → loop sends the task → model requests tools → tools execute in the sandbox → results feed back → repeat until the model stops (max 40 turns) → deliverables land in outputs/.

Step-by-Step Build Guide

Built safety-first: the sandbox before the loop, the loop before the agents, the offline tests before any live call.

PHASE 1: Sandboxed Tools

Day 1-2

Goal: An autonomous agent with shell access needs a blast radius first.

- `bash`: runs in the per-run working directory, 300s hard timeout, output capped at 20k chars so a runaway command can't flood the context.
- `write_file` / `read_file`: relative paths only; anything resolving outside the workdir is rejected.
- All errors returned as tool results (never exceptions) so the model can react and recover.

Deliverable: `execute_tool(name, input, workdir)` + `TOOL_SCHEMAS`.

PHASE 2: The Agent Loop

Day 2-4

Goal: The standard tool-use pattern, built against an interface — not a vendor client.

- Send task → read `tool_use` blocks → execute → append `tool_result` blocks → continue until `stop_reason != 'tool_use'`.
- Works with anything exposing `.messages.create(...)` — the real Anthropic client or a test fake.
- Every message and tool call printed live; max 40 turns as a hard stop.

Deliverable: `run_agent(client, system, task, workdir)` -> final message.

PHASE 3: Offline Verification

Day 4-6

Goal: Prove the loop without spending a token.

- `FakeClient` scripts a realistic sequence: write a script, run it, read its output, finish.
- Asserts: files created, script stdout returned as a tool result, final message surfaced.
- Plus: sandbox path-escape blocked, `bash` failures and missing files reported as errors, all 4 agents wired with valid task templates.

Deliverable: `python3 -m pytest tests/ -q -> 5 passed, zero API calls.`

PHASE 4: The Four Agent Definitions

Day 6-9

Goal: The cookbook discipline, retargeted at the local sandbox.

- data-analyst: narrative HTML report, 3+ plotly charts, actionable findings.
- data-scientist: dummy baseline first, stratified 5-fold CV (mean +/- std), untouched holdout, calibrated claims, limitations section.
- ai-engineer: dependency-injected LLM client (StubClient for offline evals), 10+ eval cases, retrieval hit-rate ≥ 0.8 quality gate.
- ml-engineer: mandatory self-verification — run its own pytest suite, only report success at exit code 0.

Deliverable: AGENTS registry: system prompt + default task + sample data per role.

PHASE 5: Sample Data + CLI

Day 9-11

Goal: Every agent runnable out of the box, or on your own data.

- Four generated datasets mirroring the cookbooks: sales orders, churn with planted noise features, house prices, the Acme FAQ.
- CLI: agent choice, --data, --task, --model; fresh run directory per invocation; graceful exit with instructions when no API key is set.

Deliverable: `python3 -m agentkit.run <agent> [--data ...] [--task ...]`.

Demo Results

Check	Result
Offline test suite	5 passed, 0 tokens: loop, sandbox, errors, data, wiring
Sandbox escape attempt	<code>write_file('../escape.txt')</code> -> ERROR, file not created
Bash failure handling	exit codes and timeouts returned as tool results
No-key behavior	clean exit with setup instructions, not a stack trace
Live readiness	<code>export ANTHROPIC_API_KEY</code> -> all four agents run end to end

Architecture Decisions

Why a local loop instead of the Managed Agents platform?

The notebooks target Anthropic's hosted runtime (beta access, cloud containers). The local loop needs only the standard Messages API: same prompts, same behavior, your machine as the sandbox — and the original notebooks ship in the repo for when platform access exists.

Why design the loop against an interface?

Dependency-injecting the model is what makes the framework testable: the FakeClient scripts realistic tool sequences, and if those tests pass, the only untested component is the model itself — exactly the component you shouldn't pay to test in CI. The framework practices what its AI-engineer agent preaches.

Why print every tool call?

An agent you can't watch is an agent you can't debug or trust. The stream mirrors the cookbook's event stream: every message and tool invocation, live.

Deliberately Out of Scope for v1

- Streaming token output (turn-level printing suffices to follow along).
- Parallel tool execution within a turn.
- Cost/token tracking per run (available from response.usage).
- The Managed Agents path itself — notebooks included for when access exists.

INTERVIEW TALKING POINT

"What was the hardest design decision?"

Designing the loop against an interface instead of the Anthropic client directly. It sounds small, but it's what makes the framework testable: the FakeClient plays the model with a scripted multi-turn tool sequence — write a script, run it, read the output, finish — and the tests assert the whole chain worked. If those pass, the only unverified component is the model itself, which is exactly the component I shouldn't be paying to test in CI. It's also the same dependency-injection pattern the AI-engineer agent's own system prompt demands of the code it generates. The framework practices what its agents preach.

What I'd Build Next

- Cost and token tracking per run from response.usage.
- Agent chaining: data-scientist's metrics.json feeds the ml-engineer's task.
- Streaming output and parallel tool execution.

- Same definitions on Managed Agents when platform access is available.

Companion Projects in This Portfolio

The practical twin of the Agent Orchestration System (Project 15): that project builds the coordination layer (supervisor, reviewer, human-in-the-loop) with mock agents; this one builds the execution layer — real LLM-driven agents doing real work with real tools. Together: the two halves of a production agent platform.

Repository: github.com/Anas-Amiar/Project-16-ai-data-agents